

Multiplayer UNO Card Game

Java Network Programming Project

Client-Server Architecture with Socket Programming

Project Type:	Multiplayer Card Game
Author:	Kaspian Thomas
Language:	Java
Architecture:	Client-Server
Networking:	Socket Programming
Concurrency:	Multi-threaded
Date:	March 30, 2026



Table of Contents

1.	Project Overview	3
2.	System Architecture	4
3.	Class Documentation	5
	3.1 Card Class	5
	3.2 Player Class	6
	3.3 GameRoom Class	7
	3.4 UNOServer Class	9
4.	Key Features	10
5.	Game Flow and Logic	11
6.	Network Protocol	12
7.	Concurrency and Thread Safety	13
8.	Conclusion	14

TO COMPILE:

TERMINAL 1:

```
javac *.java
java UNOServer.java
* server starts *
```

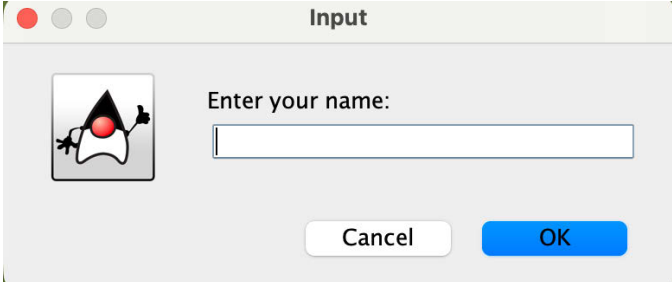
```
kast@Mac UNO % javac *.java
kast@Mac UNO % java UNOServer
UNO Server starting on port 8888
Server is running. Waiting for players..
```

TERMINAL 2:

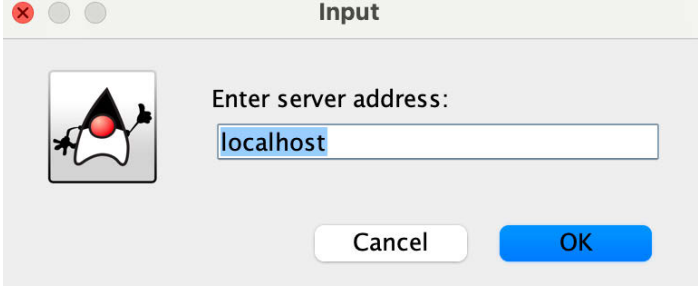
```
java UNOClient.java
*enter username*
*select localhost
```

TERMINAL 3: (OPTIONAL)

```
java UNOClient.java
*enter username*
*select localhost*
```



The screenshot shows a macOS-style dialog box titled "Input". On the left is a small icon of a character with a red nose and a black hat. To the right of the icon is the text "Enter your name:" followed by a text input field. Below the input field are two buttons: "Cancel" and "OK".



The screenshot shows a macOS-style dialog box titled "Input". On the left is a small icon of a character with a red nose and a black hat. To the right of the icon is the text "Enter server address:" followed by a text input field containing the text "localhost". Below the input field are two buttons: "Cancel" and "OK".

1. Project Overview

This project implements a fully-functional multiplayer UNO card game using Java's socket programming tools. The system allows 2-4 players to connect over a network and play the classic UNO card game in real-time. The implementation follows a client-server architecture where a central server manages game state, enforces rules, and coordinates communication between all connected players.

1.1 Key Objectives

- Implement client-server architecture using Java Socket programming
- Create a thread-safe, concurrent game server that handles multiple game rooms
- Enforce official UNO rules including card validation, special card effects, and turn management
- Provide real-time synchronization of game state across all connected clients
- Implement automated matchmaking and game initialization with countdown timers
- Support player chat functionality for an enhanced user experience

1.2 Technical Features

- **Network Communication:** TCP/IP sockets for reliable client-server communication
- **Concurrency:** Multi-threaded server using `ExecutorService` for handling multiple clients
- **Synchronization:** Thread-safe game state management with synchronized methods
- **Serialization:** Card objects implement `Serializable` for network transmission
- **Game Logic:** Complete UNO rule implementation including all special cards
- **Matchmaking:** Automatic player matching and game room creation

2. System Architecture

The UNO game follows a centralized client-server architecture. The server maintains authoritative game state and handles all game logic, while clients are responsible for presenting the user interface and sending player actions to the server.

2.1 Architecture Components

Component	Responsibility	Key Classes
Server Layer	Manages connections, game rooms, and game state	UNOServer, ClientHandler
Game Logic	Enforces UNO rules and manages game flow	GameRoom, Card
Player Management	Handles player state and communication	Player
Network Protocol	Defines message format for client-server communication	String-based protocol

2.2 Communication Flow

1. Client connects to server on port 8888
2. Server creates a ClientHandler thread for the client
3. Player sends their name and is matched to a game room
4. When 2-4 players join, a countdown timer (10 seconds) starts
5. Game begins automatically when timer expires or max players (4) is reached
6. Server broadcasts game state updates to all players in the room
7. Players send actions (PLAY_CARD, DRAW_CARD, CHAT) to server
8. Server validates actions and updates game state accordingly
9. All players receive real-time updates via broadcast messages and GUI changes

3. Class Documentation

3.1 Card Class

The Card class represents a single UNO card and implements **Serializable** for network transmission. Each card has a color (RED, YELLOW, GREEN, BLUE, or WILD), a type (NUMBER, SKIP, REVERSE, DRAW_TWO, WILD, or WILD_FOUR), and a value (0-9 for number cards, -1 for special cards).

Key Features:

- Two enumerations: Color and Type for type safety
- Card validation logic via canPlayOn() method with wild card support
- Overloaded canPlayOn() method that considers current color for wild cards
- toString() and toShortString() methods for different display formats
- Getter and setter methods for all properties

```
// Card validation with wild card color tracking
public boolean canPlayOn(Card topCard, Card.Color currentColor) {
    if (this.isWild()) { return true;
// Wild cards can always be played
    }
    if (currentColor != null && currentColor != Card.Color.WILD) {
        if (this.color == currentColor) {
            return true;
        }
    }
    return (this.color == topCard.color) || (this.type == Type.NUMBER &&
topCard.type == Type.NUMBER && this.value == topCard.value) || (this.type ==
topCard.type && this.type != Type.NUMBER); }
```

3.2 Player Class

The Player class encapsulates all player-related data and provides I/O capabilities for network communication. Each player has a unique ID (UUID), a name, a hand of cards, and an associated socket connection.

Key Components:

- UUID-based unique player identification
- ArrayList to store player's hand of cards
- Socket, PrintWriter, and BufferedReader for network I/O
- Methods for adding/removing cards from hand
- sendMessage() and receiveMessage() for communication
- Automatic socket cleanup on disconnect

Field	Type	Purpose
id	String	Unique UUID identifier
name	String	Player display name
hand	List<Card>	Cards in player's hand
socket	Socket	Network connection
out	PrintWriter	Output stream for messages
in	BufferedReader	Input stream for messages

3.3 GameRoom Class

The GameRoom class is the core of the game logic. It manages a single game instance including the deck, discard pile, player turns, and game state. Each GameRoom can accommodate 2-4 players and enforces all UNO game rules.

Core Responsibilities:

- **Deck Management:** Initializes a complete 108-card UNO deck with proper distribution
- **Player Management:** Handles player joining, removal, and turn rotation
- **Game State:** Tracks current player, direction, top card, and current color
- **Turn Logic:** Manages clockwise/counter-clockwise turn progression
- **Card Validation:** Validates whether a card can be played
- **Special Cards:** Implements effects for SKIP, REVERSE, DRAW_TWO, WILD, and WILD_FOUR
- **Timer System:** Countdown timer for automatic game start
- **Broadcasting:** Sends game state updates to all players
- **Chat Support:** Broadcasts chat messages between players

Deck Composition (108 cards total):

Card Type	Quantity	Details
Number 0	4	One per color (R, Y, G, B)
Numbers 1-9	72	Two of each number per color
SKIP	8	Two per color
REVERSE	8	Two per color
DRAW TWO	8	Two per color
WILD	4	Wild card (choose color)
WILD DRAW FOUR	4	Draw 4 + choose color

Important Methods:

- `initializeDeck()` - Creates and shuffles the 108-card deck
- `addPlayer(Player)` - Adds player and manages timer/auto-start
- `startGame()` - Initializes game state and deals cards
- `playCard(Player, index, color)` - Validates and plays a card
- `drawCard(Player)` - Handles card drawing from deck
- `executeCardEffects(Card)` - Implements special card logic
- `broadcastGameState()` - Sends current state to all players
- `broadcastChat()` - Sends chat messages to all players

3.3.1 Special Card Effects

The GameRoom class implements all official UNO special card effects with proper turn management:

Card	Effect Implementation
SKIP	Advances turn once (victim is skipped), then normal turn advance
REVERSE	Toggles direction (clockwise ↔ counter-clockwise) In 2-player game, acts like SKIP
DRAW TWO	Advances to victim, adds 2 cards to their hand Victim's turn is skipped
WILD	Allows player to choose new color No additional effects
WILD DRAW FOUR	Advances to victim, adds 4 cards to their hand Player chooses new color, victim is skipped

3.3.2 Automatic Start Timer

When 2-3 players join a room, a 10-second countdown timer automatically begins. The timer stops and the game starts immediately if:

- The countdown reaches zero (automatic start with 2-3 players)
- A 4th player joins (immediate start at max capacity)
- The game is started manually
- Player count drops below 2 before game starts (timer cancelled)

The timer uses `ScheduledExecutorService` for thread-safe countdown management and broadcasts `TIMER_TICK` messages every second to keep all clients synchronized.

3.4 UNOServer Class

The UNOServer class is the entry point of the application. It manages incoming client connections and coordinates multiple game rooms concurrently. The server uses a thread pool to handle multiple clients simultaneously without blocking.

Server Architecture:

- **Main Thread:** Listens on port 8888 for incoming connections
- **Thread Pool:** CachedThreadPool creates threads on-demand for each client
- **ClientHandler:** Inner class that handles individual client communication
- **Game Rooms:** ConcurrentHashMap stores active game rooms by room ID
- **Player Handlers:** ConcurrentHashMap maps players to their handlers
- **Matchmaking:** Automatic room assignment and creation

ClientHandler Responsibilities:

1. Reads player name from initial connection
2. Creates Player object and assigns to game room
3. Enters message processing loop to handle player commands
4. Routes commands to appropriate GameRoom methods
5. Sends acknowledgments and updates back to client
6. Handles cleanup on player disconnect

Command Processing:

Command	Parameters	Action
PLAY_CARD	cardIndex, color	Validates and plays card from hand
DRAW_CARD	None	Draws a card from deck and advances turn
GET_HAND	None	Sends current hand to client
GET_STATE	None	Triggers game state broadcast
CHAT	message	Broadcasts chat message to all players

4. Key Features

4.1 Automatic Matchmaking

Players are automatically matched to available game rooms. The server maintains a pool of game rooms and assigns players to rooms that aren't full and haven't started. If no suitable room exists, a new one is created automatically.

4.2 Real-time State Synchronization

The server broadcasts game state updates after every action. All clients receive:

- Current player's turn
- Top card on discard pile
- Current color (especially important after wild cards)
- All players' hand sizes
- Turn notifications (YOUR_TURN messages)

4.3 Complete UNO Rules Implementation

The game enforces official UNO rules including:

- Card matching by color, number, or type
- Wild cards can be played anytime and change the current color
- Special card effects (Skip, Reverse, Draw Two, Wild Draw Four)
- Proper turn rotation (clockwise/counter-clockwise)
- Win condition detection (first player to empty their hand)
- Deck reshuffling when cards run out

4.4 Thread-Safe Game State

All game state modifications use synchronized methods to prevent race conditions. The ConcurrentHashMap data structures ensure thread-safe access to game rooms and player handlers even with multiple concurrent games running.

4.5 In-Game Chat System

Players can send chat messages that are broadcast to all players in their game room. The chat system uses a simple protocol that preserves message content including special characters.

4.6 Game State Persistence

The `GameRoom` class includes a `saveGameState()` method that writes the current game status to a file. This includes the room ID, current player, direction, current color, top card, and all player hand sizes.

5. Game Flow and Logic

5.1 Connection and Game Start

Step 1: Player Connection

- Client connects to server on port 8888
- Server accepts connection and creates ClientHandler thread
- Client sends player name
- Server creates Player object with unique UUID

Step 2: Room Assignment

- Server searches for available game room
- If found: player is added to existing room
- If not found: new room is created and player joins
- Server sends JOINED_ROOM message with room ID

Step 3: Waiting for Players

- Room waits for minimum 2 players
- When 2nd player joins: 10-second countdown timer starts
- Timer broadcasts TIMER_TICK every second
- When 3rd player joins: timer resets to 10 seconds
- When 4th player joins: timer stops, game starts immediately

Step 4: Game Initialization

- Each player receives 7 cards
- First non-wild card is flipped to discard pile
- Current color is set to first card's color
- First player is randomly determined (player at index 0)
- All players receive GAME_STATE and YOUR_HAND messages
- First player receives YOUR_TURN notification

5.2 Gameplay Loop

Player Turn:

1. Player receives YOUR_TURN notification
2. Player has two options:
 - a. Play a valid card: PLAY_CARD|cardIndex|chosenColor
 - b. Draw a card: DRAW_CARD
3. Server validates the action
4. If valid: game state is updated
5. If invalid: error message sent, player retains turn
6. Turn advances to next player
7. New game state is broadcast to all players

Card Play Validation:

- Must be player's turn

- Card must match top card's color OR
- Card must match top card's number (for number cards) OR
- Card must match top card's type (for special cards) OR
- Card must be a wild card (always valid)
- After wild card: new color must be chosen

Special Card Processing:

- SKIP: Next player is skipped
- REVERSE: Direction changes (or skip in 2-player game)
- DRAW_TWO: Next player draws 2 cards and is skipped
- WILD: Current player chooses new color
- WILD_DRAW_FOUR: Next player draws 4 cards, current player chooses color

5.3 Win Condition

When a player plays their last card, the following occurs:

- Server detects hand size is zero
- GAME_WON message is broadcast with winner's name
- Game state flag is set to false (game ended)
- Room remains active for potential new game
- Players can disconnect or wait for rematch

6. Network Protocol

The game uses a simple text-based protocol where messages are delimited by the pipe character (|). All communication is done through standard Socket I/O streams.

6.1 Server → Client Messages

Message	Format	Description
Welcome	WELCOME playerName	Sent on connection
Joined Room	JOINED_ROOM roomId playerCount	Player added to room
Player Joined	PLAYER_JOINED playerName totalPlayers	Broadcast when player joins
Timer Tick	TIMER_TICK seconds	Countdown update (every second)
Timer Reset	TIMER_RESET seconds	Timer restarted (new player joined)
Game State	GAME_STATE currentPlayer topCard completeGame player1:handSize, ...	Complete game state
Your Turn	YOUR_TURN	Notification that it's your turn
Your Hand	YOUR_HAND 0:cardString, 1:cardString, ...	Current hand contents
Chat	CHAT senderName message	Chat message from player
Game Won	GAME_WON winnerName	Game over announcement
Success	SUCCESS CARD_PLAYED or SUCCESS WITHDRAW	Action successful
Error	ERROR message	Error description

6.2 Client → Server Messages

Command	Format	Description
Play Card	PLAY_CARD cardIndex chosenColor	Play card at index with optional color choice
Draw Card	DRAW_CARD	Draw one card from deck
Get Hand	GET_HAND	Request current hand
Get State	GET_STATE	Request game state update
Chat	CHAT message	Send chat message to all players

6.3 Card String Format

Cards are transmitted using the `toShortString()` method which produces compact representations:

- R5 = Red 5
- G+2 = Green Draw Two
- BS = Blue Skip
- YR = Yellow Reverse
- W = Wild
- W+4 = Wild Draw Four

This compact format reduces network bandwidth and is easily parsable by clients.

7. Concurrency and Thread Safety

The UNO server is designed to handle multiple concurrent games with many simultaneous players. Thread safety is crucial to prevent race conditions and data corruption.

7.1 Threading Model

Main Thread:

- Runs the ServerSocket accept loop
- Blocks waiting for new client connections
- Spawns ClientHandler threads for each connection

ClientHandler Threads:

- One thread per connected client
- Handles all I/O for that client
- Processes commands and routes to GameRoom methods
- Runs until client disconnects

Timer Threads:

- ScheduledExecutorService manages countdown timers
- One timer per game room (when applicable)
- Broadcasts tick updates every second
- Automatically starts game when countdown reaches zero

7.2 Synchronization Mechanisms

`GameRoom.addPlayer()` - Prevents concurrent modifications to player list

Synchronized Methods: Ensures game starts only once

- `GameRoom.playCard()` - Atomic card play and state update
- `GameRoom.drawCard()` - Thread-safe card drawing
- `GameRoom.removePlayer()` - Safe player removal

Concurrent Collections:

- `ConcurrentHashMap<String, GameRoom>` - Thread-safe game room storage
- `ConcurrentHashMap<Player, ClientHandler>` - Thread-safe player-handler mapping

ExecutorService:

- `Executors.newCachedThreadPool()` - Manages ClientHandler threads
- Automatically scales thread pool based on demand
- Reuses idle threads for efficiency

7.3 Critical Sections

The following operations require atomic execution to maintain consistency:

- **Player joining:** Check room capacity, add player, potentially start timer, broadcast update
- **Card playing:** Validate card, remove from hand, add to discard, execute effects, advance turn
- **Game start:** Deal cards, flip first card, set initial state, broadcast to all
- **Timer countdown:** Check game state, decrement counter, broadcast tick or start game
- **Player disconnect:** Remove from room, check minimum players, potentially end game

8. Conclusion

This UNO game implementation demonstrates a robust client-server architecture built with Java socket programming. The project successfully implements:

- A complete multiplayer game supporting 2-4 concurrent players
- Full UNO rules including all special cards and their effects
- Thread-safe concurrent game management with multiple simultaneous game rooms
- Real-time state synchronization across all connected clients
- Automatic matchmaking and game initialization with countdown timers
- In-game chat system for player communication
- Proper error handling and graceful disconnection management
- Clean separation of concerns with well-defined classes and responsibilities

8.1 Technical Highlights

Network Programming: Effective use of Java Sockets, PrintWriter, and BufferedReader for reliable client-server communication.

Concurrency: Thread-safe design using synchronized methods, ConcurrentHashMap, and ExecutorService for scalable multi-client handling.

Game Logic: Comprehensive implementation of UNO rules with proper validation, special card effects, and turn management.

Protocol Design: Simple, extensible text-based protocol that's easy to debug and maintain.

State Management: Centralized authoritative game state on the server prevents cheating and ensures consistency.

8.2 Potential Enhancements

While the current implementation is fully functional, several enhancements could be added:

- Graphical User Interface (GUI) using JavaFX or Swing
- Player authentication and persistent user accounts
- Game statistics and leaderboard tracking

- Reconnection support for disconnected players
- Spectator mode for watching ongoing games
- Announce UNO when one card remains in player's hand
- Custom game rules (house rules, speed UNO variants)
- Tournament mode with bracket management
- Mobile client support (Android/iOS)
- Advanced matchmaking with skill-based matching
- Game replay and analysis feature

— End of Report —